Appendix A Benchmark Dataset

A.1 IMDB Benchmark

A.1.1 Dataset Description. The IMDB dataset is designed to simulate a movie discovery and recommendation environment. It allows for querying films based on a combination of categorical metadata (e.g., genres, ratings, release years) and rich textual profiles. To set up a scenario facilitating cross-era movie discovery (e.g., finding modern successors to classic cinema), we partition the data into two relational tables, T_1 and T_2, representing different time periods. Each record encapsulates a comprehensive movie profile, including a synthesized crew description and high-dimensional embeddings for its plot and title.

A.1.2 Data Source and Construction. To construct this benchmark, we integrated official metadata with unstructured content:

- **Data Sourcing:** We extracted structured attributes (title, rating, crew identifiers) from the official IMDB non-commercial datasets. To ensure a high-quality corpus, we limited the scope to non-adult feature films released between 1950 and 2020. This was augmented with detailed plot descriptions sourced from the `jquigl/imdb-genres` collection.
- **Profile Construction:** We synthesized an `actor_director` textual field for each movie by joining the crew identifiers with the name basics table, resolving top-billed cast members and directors into human-readable strings. The final consolidated table, consisting of 85,615 cleaned records, was merged on the composite key of movie title and release year.
- **Vectorization / Partition:** We generated 1536-dimensional embeddings for the `title`, `plot`, and `actor_director` fields using the Gemini-embedding-001 model. Finally, the dataset was split by the median release year (2007) into table `imdb_t1` (42,378 records, pre-2007) and table `imdb_t2` (43,237 records, 2007–2020) to provide physical targets for relational semantic joins.

A.1.3 Workload Construction. To systematically evaluate the execution strategies, we generate a synthetic workload suite that instantiates the three core dimensions of our multistep reasoning query model: structured predicates ($\mathcal{P}$), multi-vector scoring ($\mathcal{S}$), and semantic join conditions ($\mathcal{J}$).

Predicates and Scoring ($\mathcal{P}$ and $\mathcal{S}$). We instantiate hard logical constraints ($\mathcal{P}$) using the exact relational metadata, such as bounding the release years or applying exact-match filters on categorical genres. The multi-dimensional ranking objective ($\mathcal{S}$) is formulated by aggregating the semantic similarities between user-provided query anchors and the movies' high-dimensional `plot` and `title` embeddings.

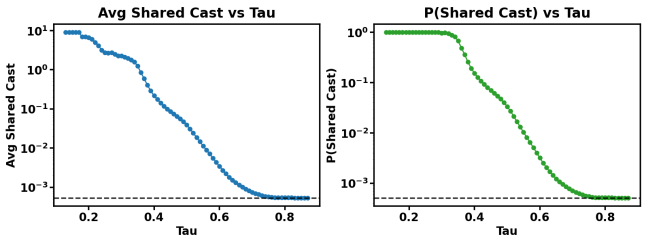


Figure 9: Validation of the semantic join proxy: shared cast metrics versus embedding distance τ. Dashed lines indicate global baselines.

Semantic Joins ($\mathcal{J}$). To evaluate inter-entity synthesis (workloads W5–W8), we simulate a cross-era discovery scenario that links classic films in `imdb_t1` with their modern counterparts in `imdb_t2`. This linkage is fundamentally driven by a semantic join over the `actor_director` embeddings.

To validate that the geometric distance (τ) in this specific embedding space serves as a robust proxy for real-world semantic relatedness, we profiled the pairwise distances across the dataset. As illustrated in Figure 9, we measured both the average number of shared cast members (left) and the probability of two movies sharing at least one cast member (right) as a function of the distance threshold τ. Note that the y-axes are scaled logarithmically, and the horizontal dashed lines denote the global expected baselines across the unconstrained Cartesian product.

The empirical results exhibit a pronounced correlation: at tight distance thresholds (small τ), both the expected density and the probability of shared cast members increase exponentially, vastly outperforming the global average. This observation perfectly aligns with our geometric premise that meaningful semantic affinities manifest strictly as extreme tail events. Consequently, this justifies our methodology of utilizing the `actor_director` embedding distance as the underlying relational proxy to construct and evaluate ground-truth semantic join pairs.

W1: Find movies by a plot description (Semantic Retrieval). To establish a performance baseline, we synthesize a query set of 100 seed phrases simulating authentic user search behaviors. We generate these search intents using OpenAI GPT-4o (temperature = 0.8) with the following exact prompt: *"Generate exactly [N] short phrases that someone might type when searching for movies. Examples: 'thriller with twist', 'romantic comedy in Paris'. Output one phrase per line, no numbering, no other text."* Each generated phrase is then vectorized into 1536-dimensional embeddings using the Gemini-embedding-001 model. To ensure uniform coverage across the partitioned corpus, the generator randomly assigns each query to one of the four possible target spaces: either the `plot` or `title` embeddings within `imdb_t1` or `imdb_t2`. Each query is parameterized with $K = 20$. This workload represents the simplest form of single-signal semantic retrieval, providing the ground-truth baseline for the system's raw vector search throughput without structural constraints.

```
-- Searching for "a thriller with unexpected twist and
    mystery"
SELECT tconst, title, plot
FROM imdb_t1
ORDER BY plot_emb <-> '[0.0336, -0.0163, ...]'
LIMIT 20;
```

Listing 2: Example W1 Query

W2: Find movies by a plot description, within genre/era (Filtered Retrieval). Building upon W1, workload W2 introduces exact structured predicates ($\mathcal{P}$) to evaluate filtered semantic search. To construct this workload, we keep the same query-text prompt

setting as W1, generate 100 intents, and vectorize each into a 1536-dimensional embedding. We then augment each query with 1 to 3 relational filters applied strictly to target-table metadata. The predicate space is bounded by three explicit rules: (1) temporal constraints applying $\geq$ or $\leq$ operators on `year` within $[1980, 2020]$; (2) quality thresholds applying the $\geq$ operator on `rating` within $[1.0, 10.0]$; and (3) categorical constraints applying the IN operator over random subsets of 12 predefined cinematic `genres`. Paired with a uniformly selected target semantic space (either `plot_emb` or `title_emb` within `imdb_t1` or `imdb_t2`) and parameterized with $K = 20$, this workload benchmarks hybrid execution with relational pruning plus dense vector scans.

```
-- Q: "a thriller with unexpected twist and mystery"
-- Filtered by specific categorical genres
SELECT tconst, title, genre, plot
FROM imdb_t1
WHERE genre IN ('Action', 'Fantasy', 'Thriller')
ORDER BY plot_emb <-> '[-0.007, -0.028, ...]'
LIMIT 20;
```

Listing 3: Example W2 Query

W3: Find movies matching a multi-aspect semantic profile (Multi-scoring retrieval). To evaluate multi-signal aggregation (Multi-$\mathcal{S}$) over a single entity without structural predicates ($\mathcal{P} = \emptyset$), workload W3 synthesizes user intents spanning multiple semantic dimensions. We generate 100 queries uniformly targeting either `imdb_t1` or `imdb_t2`. Query text generation uses a 3-slot prompt template (plot description | title style | actor/director vibe), then maps each slot to `plot_emb`, `title_emb`, and `actor_director_emb`, respectively. Each component phrase is embedded with Gemini-embedding-001 (1536-d). Ranking uses a late-binding weighted sum over these L_2 vector distances, with per-query weights normalized to 1.0 and $K = 20$.

```
-- Q: Find movies in imdb_t1 matching a multi-aspect
     profile:
-- famous thriller actors AND unexpected twist plot motif
SELECT tconst, title, year
FROM imdb_t1
ORDER BY
  0.60 * (actor_director_emb <-> '[-0.033, -0.031, ...]')
  -- demographic cast intent mapping (L2 distance)
+ 0.40 * (plot_emb <-> '[-0.007, -0.028, ...]')
  -- narrative plot intent mapping (L2 distance)
LIMIT 20;
```

Listing 4: Example W3 Query

W4: Find movies matching a multi-aspect semantic profile within strict relational filters (Filtered multi-scoring). Building upon W3, workload W4 introduces exact structural predicates ($\mathcal{P}$) to evaluate filtered multi-signal aggregation (Multi-$\mathcal{S}$) over a single entity without cross-table joins ($\mathcal{J} = \emptyset$). We synthesize 1000 queries uniformly targeting either `imdb_t1` or `imdb_t2`. Query text generation follows the same three-slot semantics as W3, using a tab-separated triple format to improve parsing robustness at larger scale. The three text slots are mapped to `plot_emb`, `title_emb`, and `actor_director_emb`, and each is embedded with Gemini-embedding-001 (1536-d).

To instantiate the structured predicates, each query is dynamically augmented with 1 to 2 relational filter groups applied strictly to the target table's metadata. The predicate generation space is strictly bounded by three explicit logical rules: (1) temporal constraints applying the $\geq$ operator on `year` sampled within $[1983, 2008]$, or applying both $\geq$ and $\leq$ operators bounded by intervals $[1983, 2003]$ and $[1995, 2018]$; (2) quality thresholds applying the $\geq$ operator on `rating` sampled within $[3.5, 7.5]$, or both $\geq$ and $\leq$ operators bounded by $[3.5, 6.5]$ and $[5.0, 9.0]$; and (3) categorical constraints applying the IN operator over random subsets of 1 to 4 predefined cinematic `genres` from a pool of 12 options. Logically, the query formulates a top-K ($K = 20$) retrieval where the final ranking is determined by a late-binding, weighted sum of the L_2 distances across 2 to 3 corresponding continuous vector spaces. Each component signal is assigned a randomized interpolation weight summing to 1.0. This workload benchmarks the capability of the database engine to efficiently execute hybrid queries combining diverse relational filters with dense multi-stream vector scans.

```
-- Q: Find movies in imdb_t2 matching a multi-aspect
     profile:
-- "enchanting romance film" (title), "magical romance
     movie stars" (cast),
-- and "romantic fantasy involving mythical realms" (plot
     )
SELECT tconst, title, year, genre
FROM imdb_t2
WHERE year >= 2005
  AND genre IN ('Adventure', 'Mystery')
ORDER BY
  0.440 * (plot_emb <-> '[-0.028, -0.012, ...]')
  -- narrative plot intent (L2 distance)
+ 0.426 * (actor_director_emb <-> '[-0.039, -0.034, ...]'
     )
  -- demographic cast intent (L2 distance)
+ 0.134 * (title_emb <-> '[-0.058, -0.016, ...]')
  -- stylistic title intent (L2 distance)
LIMIT 20;
```

Listing 5: Example W4 Query

W5: Pair up similar-cast films that fit a plot or title theme (Semantic join). Transitioning to Inter-Entity Synthesis ($\mathcal{J}$), workload W5 evaluates semantic joins without structural predicates ($\mathcal{P} = \emptyset$). At a high level, this query pattern represents a cross-era discovery task: it finds pairs of classic and modern movies that share a highly identical demographic cast or crew, and ranks these linked pairs globally based on how well one of the movies matches a specific narrative or stylistic search intent. To construct this workload, we synthesize 100 movie search intents using OpenAI GPT-4o (temperature = 0.8) with the following exact prompt: *"Generate exactly [N] short phrases that someone might type when searching for movies. Examples: 'thriller with twist', 'romantic comedy in Paris'. Output one phrase per line, no numbering, no other text."* Each generated phrase is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model.

Logically, the query formulates a distance-threshold ϵ-join on the `actor_director_emb` vectors between the `imdb_t1` and `imdb_t2` tables. To comprehensively evaluate the engine across varying linkage densities, the parameter τ is strictly sampled from predefined discrete semantic similarity tiers $(0.40, 0.45, 0.50, 0.55, 0.60)$. These distinct values map directly to physical real-world relationships: a tight threshold ($\tau = 0.40$) captures highly identical cast combinations yielding an intermediate join selectivity of 0.01%, a medium

threshold ($\tau = 0.50$) captures similar associations yielding 0.23% selectivity, while a relaxed threshold ($\tau = 0.60$) captures loosely related connections yielding an exponentially larger 5.13% intermediate join selectivity. The semantic scoring signal ($\mathcal{S}$) is uniformly assigned to target either `plot_emb` or `title_emb` on either the `imdb_t1` or `imdb_t2` table. Crucially, to prevent a single highly-ranked anchor film from dominating the result set with multiple join partners, the valid joined pairs are explicitly deduplicated by the primary key (`tconst`) of the scored table. The pairs are then globally ranked by the L_2 distance between the assigned target embedding and the query vector, retaining the overall top $K = 20$ distinct pairs.

```
-- Q: Find classic-modern movie pairs with similar cast/
     crew (L2 <= 0.40),
-- ensure distinct classic movies, globally ranked by how
      well t1 fits a plot intent
SELECT t1_tconst, t1_title, t2_title
FROM (
  SELECT DISTINCT ON (t1.tconst)
         t1.tconst AS t1_tconst,
         t1.title AS t1_title,
         t2.title AS t2_title,
         t1.plot_emb <-> '[-0.007, -0.028, ...]' AS score
  FROM imdb_t1 t1
  JOIN imdb_t2 t2
    ON (t1.actor_director_emb <-> t2.actor_director_emb)
     <= 0.40
    -- cast similarity join (sampled from discrete
     similarity tiers)
  ORDER BY t1.tconst, score
) distinct_pairs
ORDER BY score
LIMIT 20;
```

Listing 6: Example W5 Query

W6: Pair up similar-cast films that fit a plot theme, within genre/era on each side (Filtered join). Building upon W5, workload W6 introduces exact structural predicates ($\mathcal{P}$) on both sides of the semantic join to evaluate filtered inter-entity synthesis. At a high level, this query pattern represents a constrained cross-era discovery task: it finds pairs of classic and modern movies that share a highly identical demographic cast or crew, satisfying rigorous structural constraints on both sides (restricting release eras or demanding specific minimum ratings), and ranks these linked pairs globally based on how well the primary movie matches a specific narrative search intent. To construct this workload, we synthesize 100 movie plot motifs using OpenAI GPT-4o (temperature = 0.8) with the following exact prompt: *"Generate exactly [N] short phrases describing a specific movie theme, setting, or plot motif. Examples: 'a detective solving a murder in a small town', 'spaceship crew encountering an alien anomaly'. Output one phrase per line, no numbering, no other text."* Each generated phrase is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model to serve as the ranking intent on `plot_emb`.

To instantiate the structured predicates, each query is dynamically augmented with relational filters applied strictly to the metadata of both `imdb_t1` and `imdb_t2`. The predicate generation space is strictly bounded by explicit logical rules: temporal constraints applying the $\geq$ or $\leq$ operators on `year` sampled within the interval $[1980, 2020]$; quality thresholds applying the $\geq$ or $\leq$ operators on `rating` sampled within $[1.0, 10.0]$; and categorical constraints applying the IN operator over random subsets of 12 predefined cinematic `genres`. The join threshold tiers, deduplication policy, and sparse-to-dense linkage settings follow W5. W6 adds only dual-sided relational filtering before final ranking, and then ranks surviving pairs by the L_2 distance between `imdb_t1.plot_emb` and the query vector (top $K = 20$).

```
-- Q: Find thriller plot movies in imdb_t1 linked to
     imdb_t2 movies with
-- similar cast/crew (L2 <= 0.45), satisfying specific
     year and rating filters
SELECT t1_tconst, t1_title, t2_title
FROM (
  SELECT DISTINCT ON (t1.tconst)
         t1.tconst AS t1_tconst,
         t1.title AS t1_title,
         t2.title AS t2_title,
         t1.plot_emb <-> '[-0.007, -0.027, ...]' AS score
  FROM imdb_t1 t1
  JOIN imdb_t2 t2
    ON (t1.actor_director_emb <-> t2.actor_director_emb)
     <= 0.45
    -- cast similarity join (sampled from discrete
     similarity tiers)
  WHERE t1.year >= 1985
    AND t2.rating >= 5.7
  ORDER BY t1.tconst, score
) distinct_pairs
ORDER BY score
LIMIT 20;
```

Listing 7: Example W6 Query

W7: Pair up similar-cast films across two plot themes (Multi-scoring join). Building upon W5, workload W7 introduces complex multi-signal aggregation (Multi-$\mathcal{S}$) into the inter-entity synthesis pipeline ($\mathcal{J}$) without structural predicates ($\mathcal{P} = \emptyset$). At a high level, this query pattern represents a dual-intent cross-era discovery task: it finds tuples of classic and modern movies that share a highly identical demographic cast or crew, and globally ranks these pairs based on how well the primary movie matches a first narrative motif, how well the secondary movie matches a second narrative motif, and the absolute tightness of their shared cast similarity. To construct this workload, we synthesize 1000 search intents per table using OpenAI GPT-4o (temperature = 1.0) with the following exact prompt: *"Generate exactly [N] short, diverse phrases (3-8 words each) that someone might type when searching for movies. They should cover a wide range of genres, moods, settings, and themes. Avoid repeating themes already covered.\nOutput one phrase per line, no numbering, no other text."* Each phrase is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model to target either the `imdb_t1 plot_emb` or `imdb_t2 plot_emb`. The primary and secondary query vectors are paired dynamically across specific correlation tiers (anti, low, medium, high) to rigorously control query hardness.

The join threshold tiers and sparse-to-dense linkage settings follow W5. The key change in W7 is the ranking objective: globally valid pairs are scored by a late-binding weighted sum of three L_2 distances, combining `imdb_t1.plot_emb` to the first intent, `imdb_t2.plot_emb` to the second intent, and the cross-table `actor_`

director_emb distance. Each weight vector is sampled and normalized to 1.0, and the top $K = 20$ pairs are retained.

```
-- Q: Find classic-modern movie tuples with similar cast/
     crew (L2 <= 0.50),
-- ranked globally by a 3-way multi-scoring objective
SELECT t1.tconst AS t1_tconst,
       t2.tconst AS t2_tconst,
       ( 0.20 * (t1.plot_emb <-> '[-0.009, 0.021, ...]')
         -- primary plot intent (L2 distance)
       + 0.20 * (t2.plot_emb <-> '[-0.019, -0.008, ...]')
         -- secondary plot intent (L2 distance)
       + 0.60 * (t1.actor_director_emb <-> t2.
     actor_director_emb)
         -- actual cast similarity (L2 distance)
       ) AS score
FROM imdb_t1 t1
JOIN imdb_t2 t2
  ON (t1.actor_director_emb <-> t2.actor_director_emb) <=
      0.50
  -- cast similarity join (sampled from discrete
     similarity tiers)
ORDER BY score
LIMIT 20;
```

Listing 8: Example W7 Query

W8: Pair up similar-cast films across two plot themes, within genre/era on each side (Filtered multi-scoring join). Building upon W7, workload W8 represents the most comprehensive query pattern, introducing exact structural predicates ($\mathcal{P}$) on both sides of the multi-signal aggregation (Multi-$\mathcal{S}$) and inter-entity synthesis pipeline ($\mathcal{J}$). At a high level, this query pattern represents a constrained, dual-intent cross-era discovery task: it finds tuples of classic and modern movies that share a highly identical demographic cast or crew, satisfying rigorous structural constraints on both sides (restricting release eras or demanding specific minimum ratings), and globally ranks these linked pairs based on how well the primary movie matches a first narrative motif, how well the secondary movie matches a second narrative motif, and the absolute tightness of their shared cast similarity. To construct this workload, we synthesize 100 pairs of distinct movie concepts using OpenAI GPT-4o (temperature = 0.8) with the following exact prompt: *"Generate exactly [N] pairs of movie concepts. Each line should contain two distinct concepts separated by a pipe character '|'. For example: 'A gritty sci-fi thriller about time travel | A lighthearted comedy about a robot learning to love'. Output only the pipe-separated pairs, one pair per line."* Each text field in the generated pair is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model, with the first concept targeting the imdb_t1 plot_emb and the second targeting the imdb_t2 plot_emb.

To instantiate the structured predicates, each query is dynamically augmented with relational filters applied strictly to the metadata of both imdb_t1 and imdb_t2. The predicate generation space is strictly bounded by explicit logical rules: temporal constraints applying the $\geq$ or $\leq$ operators on year sampled within the interval $[1980, 2020]$; quality thresholds applying the $\geq$ or $\leq$ operators on rating sampled within $[1.0, 10.0]$; and categorical constraints applying the IN operator over random subsets of 12 predefined cinematic genres. W8 follows W5 for join tiers, and follows W7 for three-signal late-binding ranking; its only additional step is applying dual-sided relational filters before ranking. The final result keeps the top $K = 20$ distinct pairs.

```
-- Q: Find "Sci-fi movies with robots" in imdb_t1 (year
     >= 1985) linked to
-- "Exploration documentaries of outer space" in imdb_t2
     (rating >= 5.7)
-- with similar cast/crew (L2 <= 0.50), ranked globally
     by a 3-way objective
SELECT t1.tconst AS t1_tconst,
       t1.title AS t1_title,
       t2.title AS t2_title,
       ( 0.25 * (t1.plot_emb <-> '[-0.009, 0.021, ...]')
         -- primary plot intent (L2 distance)
       + 0.25 * (t2.plot_emb <-> '[-0.027, -0.013, ...]')
         -- secondary plot intent (L2 distance)
       + 0.50 * (t1.actor_director_emb <-> t2.
     actor_director_emb)
         -- actual cast similarity (L2 distance)
       ) AS score
FROM imdb_t1 t1
JOIN imdb_t2 t2
  ON (t1.actor_director_emb <-> t2.actor_director_emb) <=
      0.50
  -- cast similarity join (sampled from discrete
     similarity tiers)
WHERE t1.year >= 1985
  AND t2.rating >= 5.7
ORDER BY score
LIMIT 20;
```

Listing 9: Example W8 Query

A.2 MOLECULE Benchmark

A.2.1 Dataset Description. Unlike IMDB, which is positioned as a fabricated discovery benchmark, MOLECULE provides real hybrid workloads extracted from an operational Biofoundry project dedicated to academic knowledge integration and retrieval. The underlying database naturally stores heterogeneous entities—including research papers, researchers, organizations, and biochemical concepts—augmented with semantic tags, structured attributes, and multiple field-level embeddings. To expose this complex, evidence-centric biomedical reasoning in a relational benchmark, we project the relevant entity interactions into a bipartite schema: a fact-side relation (molecule_fact) and a paper-side relation (molecule_paper). This layout successfully captures the target user behavior of our deep research agent system: proposing molecule-related concepts, retrieving supporting literature, applying metadata constraints, and ranking candidates under multi-signal criteria.

A.2.2 Data Source and Construction. To construct this benchmark, we combine two sources: (i) molecule-centric fact/paper data extracted from the broader heterogeneous knowledge graph and (ii) deep research agent query traces. The construction pipeline is:

- **Agent-Trace Grounding:** We collect historical deep research agent queries and cluster them into eight workload intents (W1–W8). For each intent class x, we retain a set of representative trace templates T_x, where each template includes the query skeleton and the fields that must be instantiated (query_text, predicates, join tier, weight vector, or combinations thereof).

- **Data Sourcing:** We collect molecule facts and publication records with complete text fields and normalized molecule identifiers from the Biofoundry integration engine, then retain only rows with valid metadata required by the template instantiation fields.
- **Profile Construction:** On the fact side, each row stores a fact statement plus structured molecule descriptors; on the paper side, each row stores publication metadata and abstract-level content. This yields a physically separated but semantically linkable two-relation layout aligned with agent reasoning hops.
- **Vectorization / Structural Features:** We encode textual fields with Gemini-embedding-001 into 1536-dimensional vectors and represent molecular structure with 1024-bit ECFP fingerprints. These representations are exactly the fields consumed by template-driven augmentation from T_x.

A.2.3 Workload Construction. To systematically evaluate execution strategies, we construct eight workloads that cover the full taxonomy of our query model: structured predicates ($\mathcal{P}$), multi-signal scoring ($\mathcal{S}$), and semantic joins ($\mathcal{J}$). For each workload x, we follow an agent-grounded augmentation recipe: (1) select representative templates T_x from deep research agent traces, (2) map template slots to physical fields, (3) generate or sample slot values, and (4) materialize augmented queries as workload JSON.

Predicates and Scoring ($\mathcal{P}$ and $\mathcal{S}$). For fact-side filtering, predicate atoms are sampled from categorical fields (`fact_type`, `claim_polarity`, `mol_source_origin`, `mol_toxicity_flag`, `mol_moa`, `mol_target_type`, `evidence_type`, `organism`), numeric fields (`mol_mw`, `mol_logp`, `mol_tpsa`, `mol_num_atoms`, `mol_num_rings`, `mol_num_hbd`, `mol_num_hba`), and array containment on `mol_therapeutic_area`. Multi-signal scoring combines fact-text semantic distance and molecule-structure Jaccard distance, with per-query randomized weights normalized to sum to 1.

Semantic Joins ($\mathcal{J}$). For inter-entity synthesis (W5–W8), we link fact rows to paper rows using a distance-threshold join over textual evidence vectors. The threshold τ is sampled from discrete tiers $\{0.45, 0.50, 0.55, 0.65\}$, labeled *very_similar*, *similar*, *related*, and *loose*. These tiers induce exponentially different candidate densities (from thousands to millions of pairs), which we intentionally preserve to stress both sparse and dense join regimes.

To avoid synthetic key joins, all cross-table linkage is parameterized geometrically by τ; smaller tiers produce strict semantic evidence alignment, and larger tiers expand recall with sharply higher pair counts. This discrete-tier design mirrors realistic retrieval behavior where semantic relatedness appears as tail events in embedding space.

W1: Find facts on a research topic (Semantic retrieval). For W1, we first extract a template set T_1 from agent traces whose intent is single-hop fact retrieval without filters or joins. We then instantiate the `query_text` slot using OpenAI GPT-4o (temperature = 0.9) with the exact prompt: *"Generate exactly [N] diverse natural-language queries a biomedical researcher might type when searching a database of facts about drug molecules and proteins. Cover varied topics: drug mechanism of action, toxicity, pharmacokinetics, structure-activity relationships, clinical outcomes, natural products, therapeutic areas (oncology / neurology / infectious / metabolic / cardio / psychiatry / immunology), organisms tested (human / mouse / rat / cell_line / bacteria), in vitro vs in vivo evidence, and synthesis routes. Vary phrasing style: keyword style, question style, long-sentence style. Keep each under 15 words, realistic, no numbering, one phrase per line."* Instantiated queries are deduplicated, embedded by Gemini-embedding-001 (1536-d), and bound to `molecule_fact.fact_emb` with $K = 20$. This yields pure semantic retrieval with $\mathcal{P} = \emptyset$ and $\mathcal{J} = \emptyset$.

```
-- Q: "inhibits beta-amyloid aggregation"
SELECT id, organism, property_type, fact_description
FROM molecule_fact
ORDER BY fact_emb <-> '[-0.0125, 0.0341, ...]'
LIMIT 20;
```

Listing 10: Example W1 Query

W2: Find facts on a research topic, within organism/property scope (Filtered retrieval). For W2, we extract templates T_2 from agent traces where semantic retrieval is explicitly conditioned by metadata constraints. We instantiate `query_text` by referring to the W1 generation setting, then augment each template with sampled predicate slots (1–3 predicates) over `molecule_fact`. The predicate slot space is explicitly bounded: categorical IN predicates over `fact_type`, `claim_polarity`, `mol_source_origin`, `mol_toxicity_flag`, `mol_moa`, `mol_target_type`, `evidence_type`, `organism`; numeric threshold predicates ($\geq, \leq$) over `mol_mw`, `mol_logp`, `mol_tpsa`, `mol_num_atoms`, `mol_num_rings`, `mol_num_hbd`, `mol_num_hba`; and array containment (`@>`) on `mol_therapeutic_area`. We retain augmented queries by selectivity buckets: *tight* $[0.01, 0.10)$ at 30%, *medium* $[0.10, 0.30)$ at 40%, and *loose* $[0.30, 0.70)$ at 30%.

```
-- Q: "drug interaction mechanisms with cytochrome P450
    enzymes"
-- medium-selectivity bundle: numeric + categorical +
    array containment
SELECT id, organism, property_type, fact_description
FROM molecule_fact
WHERE mol_tpsa >= 115.54
  AND organism IN ('human', 'mouse')
  AND mol_therapeutic_area @> ARRAY['oncology']
ORDER BY fact_emb <-> '[-0.0102, -0.0001, ...]'
LIMIT 20;
```

Listing 11: Example W2 Query

W3: Find facts on a topic about a reference molecule (Multi-scoring retrieval). For W3, we extract templates T_3 from traces where the agent jointly reasons over textual evidence and molecule structure in one retrieval hop. We instantiate two template slots: (i) `query_text` (referring to the W1 generation setting) mapped to `molecule_fact.fact_emb` and (ii) a reference molecule slot mapped to `molecule_fact.mol_ecfp` as a 1024-bit ECFP anchor. We then instantiate per-query weight slots and normalize to 1.0, producing a late-binding weighted sum across L2 text distance and Jaccard structure distance.

```
-- Q: "drug interaction mechanisms..." related to a
    specific reference molecule
SELECT id, organism, property_type, fact_description
FROM molecule_fact
ORDER BY
  0.60 * (fact_emb <-> '[-0.0101, -0.0001, ...]')
  -- biological topic intent (semantic distance)
```

```
+ 0.40 * (mol_ecfp <%> '[0, 0, 0, 1, ...]')
  -- structural chemical similarity (Jaccard distance)
LIMIT 20;
```

Listing 12: Example W3 Query

W4: Find facts on a topic about a reference molecule, within organism/property scope (Filtered multi-scoring). For W4, we extract templates T_4 from traces that combine dual-signal scoring with explicit fact-side filtering. We instantiate text, structure, and weight slots exactly as in W3, then instantiate 1–3 predicate slots by referring to the W2 predicate setting and retain queries under the same selectivity-bucket policy (*tight* 30%, *medium* 40%, *loose* 30%). This yields a template-grounded filtered multi-scoring workload on a single relation.

```
-- Q: "drug interaction mechanisms..." related to a
     specific reference molecule
-- filtered multi-scoring: same two signals as W3 +
     predicates following W2
SELECT id, organism, property_type, fact_description
FROM molecule_fact
WHERE mol_tpsa >= 115.54
  AND evidence_type IN ('in_vitro', 'in_vivo')
  AND mol_target_type IN ('enzyme', 'receptor')
ORDER BY
  0.60 * (fact_emb <-> '[-0.0101, -0.0001, ...]')
  -- biological topic intent (semantic distance)
+ 0.40 * (mol_ecfp <%> '[0, 0, 0, 1, ...]')
  -- structural chemical similarity (Jaccard distance)
LIMIT 20;
```

Listing 13: Example W4 Query

W5: Find paper-supported facts about a reference molecule (Semantic join). For W5, we extract templates T_5 from traces where the agent retrieves fact candidates and then links them to supporting literature. We instantiate a structure-anchor slot (sampled valid `mol_id`, 1024-bit ECFP) and a join-threshold slot for `molecule_fact.fact_emb` ↔ `molecule_paper.abstract_emb`. The join-threshold slot is sampled from discrete tiers: `very_similar` (0.45), `similar` (0.50), `related` (0.55), `loose` (0.65), and `unrelated` (0.75), preserving the observed sparse-to-dense evidence-linking regimes in agent behavior.

```
-- Q: Find facts matching a specific molecular structure,
-- then join papers providing semantic textual evidence (
     L2 <= 0.45)
SELECT fact_id, paper_id
FROM (
  SELECT DISTINCT ON (f.id)
         f.id AS fact_id,
         p.pmid AS paper_id,
         (f.fact_emb <-> p.abstract_emb) AS join_dist
  FROM (
    SELECT id, fact_emb
    FROM molecule_fact
    ORDER BY mol_ecfp <%> '[0, 0, 0, 0, ...]'  -- fact-
     side structure intent
    LIMIT 20
  ) AS f
  JOIN molecule_paper AS p
    ON (f.fact_emb <-> p.abstract_emb) <= 0.45   --
     very_similar tier
  ORDER BY f.id, join_dist
) AS dedup
ORDER BY join_dist
LIMIT 20;
```

Listing 14: Example W5 Query

W6: Find paper-supported facts about a reference molecule, within fact/paper filters (Filtered join). For W6, we extract templates T_6 from traces where evidence-linking is constrained by both fact-side and paper-side metadata. We instantiate W5 structure-anchor and join slots, then instantiate dual-sided predicate slots using an oversampled candidate pool: fact-side predicates by referring to the W2 predicate setting and paper-side predicates over `year`, `journal`, `pub_type`, and `mesh_terms`. Candidate templates are retained via soft selectivity acceptance (knee at 0.10), producing a broad left/right selectivity grid aligned with agent-side constrained retrieval behavior.

```
-- Q: Facts for a reference molecule (natural/microbial
     origin),
-- joined with US Gov supported papers providing semantic
     evidence
SELECT fact_id, paper_id
FROM (
  SELECT DISTINCT ON (f.id)
         f.id AS fact_id,
         p.pmid AS paper_id,
         (f.fact_emb <-> p.abstract_emb) AS join_dist
  FROM (
    SELECT id, fact_emb
    FROM molecule_fact
    WHERE mol_source_origin IN ('natural_product', '
     microbial')
      AND mol_toxicity_flag IN ('non_toxic')
    ORDER BY mol_ecfp <%> '[0, 0, 0, 0, ...]'
    LIMIT 20
  ) AS f
  JOIN molecule_paper AS p
    ON (f.fact_emb <-> p.abstract_emb) <= 0.45
   AND p.pub_type @> ARRAY['Research Support, U.S. Gov''t
     ']
   AND p.mesh_terms @> ARRAY['Humans']
  ORDER BY f.id, join_dist
) AS dedup
ORDER BY join_dist
LIMIT 20;
```

Listing 15: Example W6 Query

W7: Find paper-supported facts combining a reference molecule, a topic, and tight fact-paper alignment (Multi-scoring join). For W7, we extract templates T_7 from traces where the agent performs cross-table tuple retrieval with three simultaneous ranking signals. We instantiate a molecule-anchor slot, a topic-text slot, a join-tier slot, and a three-weight slot. Topic texts are generated by OpenAI GPT-4o (temperature = 0.8) from four exact prompt templates (`mol_specific`, `area_specific`, `target_topic`, `unrelated`) and embedded by Gemini-embedding-001 onto `molecule_paper.abstract_emb`. Join tiers are sampled from $\{0.45, 0.50, 0.55, 0.65\}$ and weight slots are normalized to 1.0, yielding a template-grounded late-binding multi-signal join workload for global tuple pairs.

```
-- Q: Find fact-paper tuples matching a molecular
     structure and topic,
-- ranked globally by a 3-way objective
SELECT f.id AS fact_id,
       p.pmid AS paper_id,
```

```
       ( 0.35 * (f.mol_ecfp <%> '[0, 1, 0, ...]')      -- fact molecule intent
       + 0.33 * (p.abstract_emb <-> '[-0.009, 0.021, ...]') -- paper topic intent
       + 0.32 * (f.fact_emb <-> p.abstract_emb) ) AS score  -- join penalty
FROM molecule_fact AS f
JOIN molecule_paper AS p
  ON (f.fact_emb <-> p.abstract_emb) <= 0.50  -- similar tier
ORDER BY score
LIMIT 20;
```

Listing 16: Example W7 Query

W8: Find paper-supported facts combining molecule/topic/fact-paper alignment, within fact/paper filters (Filtered multi-scoring join). For W8, we extract templates T_8 from traces representing full deep-research behavior: constrained cross-entity tuple linking with multi-signal ranking. We instantiate W7 slots (molecule anchor, topic embedding, join tier, normalized three-weight vector) and W6 slots (dual-sided predicate bundle) together. Operationally, we pair an oversampled W6 predicate pool with W7 query templates, then select a final subset that preserves diversity over filter hardness, join density tier, and topic-to-molecule correlation tier. This yields the comprehensive agent-grounded tuple workload where $\mathcal{P}$, $\mathcal{S}$, and $\mathcal{J}$ are jointly active.

```
-- Q: Find tuples of Catalyst facts + NIH-funded papers, ranked
-- globally by molecule intent + topic intent + alignment penalty
SELECT f.id AS fact_id,
       p.pmid AS paper_id,
       ( 0.35 * (f.mol_ecfp <%> '[0, 1, 0, ...]')
       + 0.33 * (p.abstract_emb <-> '[-0.006, 0.012, ...]')
       + 0.32 * (f.fact_emb <-> p.abstract_emb) ) AS score
FROM molecule_fact AS f
JOIN molecule_paper AS p
  ON (f.fact_emb <-> p.abstract_emb) <= 0.55   -- related tier
WHERE f.mol_moa IN ('catalyst')
  AND f.evidence_type IN ('in_vivo', 'clinical')
  AND p.pub_type @> ARRAY['Research Support, N.I.H., Extramural']
  AND p.year >= 2005
ORDER BY score
LIMIT 20;
```

Listing 17: Example W8 Query

Appendix B Integration with Semantic Operator Systems

As established in Sec. 2.1, DASE is designed to assist *semantic operator systems* [37, 43, 47] by acting as a cost-effective prefiltering stage that prunes input data before any LLM invocation. These systems typically implement a suite of LLM-backed *semantic operators*, including SEM_FILTER, SEM_CLASSIFY, SEM_RANK, SEM_JOIN, and SEM_MAP. DASE functions as a plug-in prefilter for these operators, leveraging multi-facet vector embeddings to identify the most promising candidate tuples for downstream LLM evaluation. Detailed SemBench performance results are provided in Table 3.

B.1 The Prefilter Pipeline

In this architecture, DASE serves as a pluggable prefilter component. The overall workflow typically operates in two stages: DASE first recommends a high-recall candidate set, which a full-featured semantic operator engine subsequently evaluates and refines using targeted LLM calls. Alternatively, in scenarios where the downstream semantic operator is omitted, the workflow bypasses LLM evaluation entirely and simply accepts all of DASE's multi-dimensional recommendations directly as the final output. This "DASE alone" configuration (denoted as *DASE* in Table 3) remains highly competitive whenever the embedding distance effectively separates relevant tuples from the background distribution.

B.2 Per-Operator Strategies

The integration logic is specifically tailored to the semantics of each operator. Crucially, DASE treats every natural-language query as a *multi-dimensional evaluation task*: it decomposes the user's semantic intent into a statistical proxy comprising weighted aggregations over multiple, potentially heterogeneous embedding streams (e.g., visual, textual, and structural). At the execution layer, these operators are natively compiled into the multi-scoring architecture introduced in Sec. 4, where independent Filtered-Score Streamers (FSS) are synchronized and pruned by the Multi-Scoring Aggregator (MSA).

B.2.1 SEM_FILTER. A SEM_FILTER evaluates a natural-language Boolean predicate ℓ on an input relation T. For a query such as "finding a blue shirt with positive reviews," the semantics are not restricted to a single modality. DASE captures this by modeling the filter as a joint multi-scoring task across all relevant feature spaces $\mathcal{F}$ (e.g., image embeddings for visual traits and text embeddings for sentiment).

For each space $f \in \mathcal{F}$, DASE derives a specific set of positive anchors P_f and antithetical anchors N_f. The MSA then synchronizes these independent FSS streams to compute a cross-dimensional contrastive margin on the fly:

$$s_i = \sum_{f \in \mathcal{F}} w_f \left(\frac{1}{|P_f|} \sum_{p \in P_f} \cos(\mathbf{e}_p, \mathbf{e}_i^{(f)}) - \frac{1}{|N_f|} \sum_{n \in N_f} \cos(\mathbf{e}_n, \mathbf{e}_i^{(f)}) \right)$$

where $\mathbf{e}_i^{(f)}$ is the embedding of tuple i in space f, and w_f balances the contribution of each modality. This mechanism natively aggregates signals—such as visual blueness and textual sentiment—to identify top candidates. Because DASE acts as a high-recall prefilter, avoiding false negatives is paramount. We employ two recommendation rules that utilize intentionally relaxed boundaries:

- *Margin-based uncertainty band.* For total labeling tasks (e.g., *Wildlife* Q1), DASE applies thresholds to the joint margin s_i. To aggressively protect recall, it forwards a conservatively wide uncertain band (typically $\alpha \approx 0.2$–0.5) for LLM evaluation, while accepting the cheap sign of s_i for confidently extreme rows.
- *Top-K' shortlist.* For `LIMIT` K queries (e.g., *Movie* Q1), the MSA seamlessly streams the K' tuples with the highest aggregate s_i. The downstream engine's `AI.IF` then evaluates this multi-dimensional shortlist, short-circuiting at the K-th

valid survivor. This heuristic reduces the query latency for *Movie* Q1 from 26.3 s to 0.5 s.

B.2.2 SEM_CLASSIFY. A SEM_CLASSIFY operator assigns each tuple to one of K enumerated classes $C = \{c_1, \ldots, c_K\}$. Whereas SEM_FILTER contrasts a single positive concept against its antithetical paraphrases, SEM_CLASSIFY generalises this to a K-way contrast: each class becomes its own anchor, and the per-row decision reduces to identifying which anchor wins. For instance, in *Cars* Q10 the engine must classify automotive complaints into 24 distinct problem categories, where a complaint may comprise both a textual description and an image of the defect, requiring multi-modal reasoning.

DASE natively maps this multi-class evaluation onto a parallel multi-scoring retrieval task. It embeds each class anchor prompt c_k once per facet to obtain $\mathbf{e}_{c_k}^{(f)}$, initialising an independent FSS stream per category. The MSA then synchronises these K streams across the required feature spaces $\mathcal{F}$ (e.g., visual embeddings of the damaged car part and text embeddings of the complaint description), aggregating per-facet cosine similarities into a per-class affinity $s_i^{(k)}$, from which a proxy label $\hat{k}_i$ and confidence margin m_i follow directly:

$$s_i^{(k)} = \sum_{f \in \mathcal{F}} w_f \cos\!\left(\mathbf{e}_{c_k}^{(f)}, \mathbf{e}_i^{(f)}\right), \; k = 1, \ldots, K,$$

$$\hat{k}_i = \arg\max_k s_i^{(k)}, \; m_i = s_i^{(\hat{k}_i)} - \max_{k \neq \hat{k}_i} s_i^{(k)}.$$

All three quantities are computed without any LLM invocation. DASE then partitions the result set on m_i: the bottom-α tuples (smallest margin) are forwarded to the downstream engine for rigorous refinement via `AI.CLASSIFY`, while the engine safely accepts the cheap argmax $\hat{k}_i$ on the remaining confident rows. The final output unions the LLM-resolved tail with the DASE-asserted confident bulk, achieving macro F_1 parity with an exhaustive all-LLM baseline at roughly half the inference cost (Table 3).

B.2.3 SEM_RANK. A SEM_RANK operator orders or scores tuples according to a defined rubric. DASE compiles this requirement into two distinct multi-scoring execution paths based on the query objective:

- *Top-K retrieval.* When the query strictly demands the highest-ranking candidates, the execution naturally reduces to the SEM_FILTER + `LIMIT` paradigm of Sec. B.2.1. DASE leverages the MSA to synchronise contrastive margin streams across all targeted feature spaces $\mathcal{F}$, organically short-circuiting the retrieval once K confidently top-ranked tuples survive downstream `AI.IF` validation.
- *Rubric scoring.* When a strict numeric label is required for *every* row (e.g., *Movie* Q9, which scores from 1 to 5 how much a reviewer liked a movie), the task is structurally a SEM_CLASSIFY over an *ordinal* class set: each rubric grade $g \in \{1, \ldots, G\}$ plays the role of a class anchor c_g, and the per-grade affinity $s_i^{(g)}$, proxy label $\hat{g}_i = \arg\max_g s_i^{(g)}$, and confidence margin m_i all carry over verbatim from the formulation in Sec. B.2.2. The only operator-specific change lies in the cascade policy. Because global rank-correlation metrics like Spearman integrate over the entire score distribution, a single mis-ranked tuple can disproportionately degrade the global metric, so DASE cannot afford the aggressive accept-rate used for nominal classification. The bottom-α tail (smallest m_i) is therefore widened substantially ($\alpha \approx 0.7$) and refined via `AI.SCORE` rather than `AI.CLASSIFY`; the proxy argmax $\hat{g}_i$ is accepted only on the confident head, and the two streams are concatenated into the final ranking.

B.2.4 SEM_JOIN. A SEM_JOIN evaluates a match predicate over pairs $(t_i, t_j) \in T_1 \times T_2$, such as verifying if two items share the same brand and category (*Ecomm* Q7). Unlike the per-row operators above, the candidate space here is the Cartesian product $|T_1| \cdot |T_2|$, so prefiltering is not merely an optimisation but a feasibility requirement. DASE evaluates the match proxy directly against the SemJI, which strictly decouples continuous multi-facet similarities in `SemJI(embed)` from discrete semantic truths in `SemJI(AI.IF)`.

During execution, DASE first probes `SemJI(AI.IF)` to resolve previously verified pairs at zero LLM cost. For unresolved pairs, the MSA translates the NL match criteria into a multi-scoring problem, synchronising the traversal across multiple `SemJI(embed)` streams (e.g., product titles *and* images) to compute a joint similarity score.

- *Two-sided uncertainty band.* For joins like *Ecomm* Q7, the MSA organically identifies pairs with the highest aggregate scores ($S_{ij} \geq \tau_{\text{hi}}$) as confident matches. Pairs in the intermediate range ($\tau_{\text{lo}} < S_{ij} < \tau_{\text{hi}}$)—where combined visual and textual cues are strong but exact identity is ambiguous—are forwarded to the LLM. The MSA traversal short-circuits once the upper bound of the joint score drops below τ_{lo}, implicitly pruning the massive tail of certain negatives. If the materialised caches exhaust before reaching τ_{lo}, DASE gracefully resumes the stream by dynamically evaluating distances via the base vector indices as discussed in Sec. 3.2.
- *Top-K poles.* For queries seeking extreme pairs (e.g., *Movie* Q5), the MSA fetches the K highest- and lowest-scoring pairs by directly aggregating the underlying facet streams.

Crucially, as the downstream LLM resolves uncertain cases via `AI.IF`, the deterministic results are written back exclusively to `SemJI(AI.IF)`, effectively amortising the cost of multi-faceted reasoning for subsequent queries.

B.2.5 SEM_MAP. A SEM_MAP call applies a per-row natural-language transformation that returns a free-form string, e.g., brand extraction (*Ecomm* Q3), colour labelling (*Ecomm* Q12), or genre clustering (*MMQA* Q4). Because the output domain is open-vocabulary, neither the contrastive margin of SEM_FILTER nor the K-way argmax of SEM_CLASSIFY applies—there is no fixed anchor set to contrast against. Instead, DASE exploits a complementary form of semantic redundancy: tuples whose embeddings are near-duplicates across the query-relevant feature spaces $\mathcal{F}$ are statistically guaranteed to map to the same target string, so a single LLM call on one representative safely labels its entire neighbourhood.

To compose this joint similarity, the MSA initialises a parallel FSS per facet $f \in \mathcal{F}$ and aggregates the per-facet cosine streams into a unified affinity $\sigma_{ij} = \sum_{f \in \mathcal{F}} w_f \cos(\mathbf{e}_i^{(f)}, \mathbf{e}_j^{(f)})$ on the fly. DASE

then clusters the input tuples by agglomerative clustering on this aggregate affinity (complete linkage, distance threshold τ_d=0.10, i.e., $\sigma_{ij} \geq 0.90$) and recommends only the medoid nearest each cluster centroid. The downstream engine runs `AI.GENERATE` on this condensed K-tuple shortlist (e.g., ∼90 medoids out of 500 rows on *Ecomm* Q3), and the returned label is broadcast to the entire cluster via a SQL join on the synthetic cluster ID. Because clusters form in the joint multi-facet space rather than over any single embedding axis, the medoid is robust to single-modality drift—visually similar but textually distinct items remain in separate clusters. Quality degrades only when τ_d is lax enough to fuse genuinely distinct entities, which on *Ecomm* costs less than 0.01 ARI while reducing LLM invocations by an order of magnitude.